**Manual del Programador Actividad 7 del 2do Cuatrimestre**
**Proyecto: Mini Juego “Piedra, Papel o Tijera” con persistencia en Disco**
**Materia: Laboratorio de Programación**
**Institución: E.E.S.T. N 1 “Eduardo Ader” Vicente López**

## 1.Descripción General del Sistema

Este programa en C++ está diseñado para resolver uno de los problemas clásicos de los programadores, y es diferenciar entre la memoria RAM (volátil, que se borra al cerrar el programa) y el almacenamiento secundario o disco(persistente, donde los datos quedan guardados en un archivo de texto).
El sistema permite jugar al clásico Piedra, Papel o Tijera contra la Inteligencia Artificial basada en números aleatorios, registrar cada jugada con una estructura de datos y guardarla automáticamente en formato CSV dentro de un archivo de texto (historial_partidas.txt) para poder consultarla más adelante, incluso si se cierra la consola por completo.

## 2. Explicación de Componentes Clave del Código

a) **Modelado de Datos con struct (struct Partida)**
Para no tener que manejar cinco variables sueltas cada vez que se juega una partida (lo cual sería difícil de mantener y desordenado), se agrupa toda la información relacionada en una sola entidad estructurada llamada Partida:

```
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

- Se usa porque nos permite empaquetar datos heterogéneos (números enteros y cadenas de texto) en un solo bloque lógico. De esta forma, podemos pasar una partida entera como parámetro a las funciones (por ejemplo, guardarEnDisco(const Partida& p)) de forma limpia y ordenada.

b) **Persistencia y Estructura en Disco (ofstream y ios::app)**
La función guardarEnDisco se encarga de abrir el archivo de texto y volcar los datos de la última partida jugada:

```
void guardarEnDisco(const Partida& p) {
    // ios::app permite agregar al final sin borrar partidas anteriores
    ofstream archivo("src/historial_partidas.txt", ios::app);

    if (!archivo.is_open()) {
        cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
```

```
        return;
    }

    // Guardado de datos separados por comas (Formato CSV)
    archivo << p.idPartida << ","
            << p.jugador << ","
            << p.eleccionJugador << ","
            << p.eleccionIA << ","
            << p.resultado << endl;

    archivo.close();
    cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." <<
endl;
}
```

- **El detalle clave (ios::app):** Si se abriera el archivo con un ofstream común sin el parámetro ios::app, cada vez que guardemos una partida se sobreescribirá el archivo completo y perdemos el historial anterior. Gracias al modo Append (ios::app), el sistema operativo se encarga de añadir los nuevos registros justo al final del archivo de texto existente.
- **Formato CSV:** Los datos se separan con comas ( , ) para facilitar tanto su escritura como su posterior lectura secuencial.

**c) Lectura del Historial desde el Disco (ifstream y getline)**

Para recuperar los datos almacenados en el archivo de texto y mostrarlos en pantalla dentro del menú, se utiliza un flujo de entradas (ifstream):

```
void leerHistorialDisco() {
ifstream archivo("src/historial_partidas.txt");

if (!archivo.is_open()) {
cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
return;
}

string idStr, jugador, elJug, elIA, resultado;
int total = 0;

// Lectura secuencial delimitada por comas
while (getline(archivo, idStr, ',') &&
       getline(archivo, jugador, ',') &&
       getline(archivo, elJug, ',') &&
       getline(archivo, elIA, ',') &&
       getline(archivo, resultado)) {

 total++;
 cout << "Partida #" << idStr
       << " | Jugador: " << jugador
```

```
            << " | " << elJug << " vs " << elIA
            << " | Resultado: " << resultado << endl;
    }

    archivo.close();
     // ...
}
```

- La función recorre el archivo línea por línea utilizando la función getline delimitada por comas. Extrae ordenadamente el ID, el nombre del jugador, la jugada del usuario, la de la IA y el resultado final, permitiendo reconstruir los objetos en pantalla sin importar cuantas veces se haya cerrado el programa.

**d) Gestión del Buffer de Entrada (cin.ignore() )**

Dentro de la función jugarPartida, nos encontramos con esta línea antes de pedir el nombre del usuario:

```
cout << "Ingrese su Nombre / Nickname: ";
cin.ignore(); // Limpieza de búfer
getline(cin, p.jugador);
```

- **Para qué sirve:** Cuando el usuario interactúa con la consola y presiona cin >> opción o ingresa un número, queda guardado en el buffer un carácter de salto de línea (\n o el “Enter”). Si después intentamos usar getline para leer una cadena con espacios o texto completo, este se tragaría ese Enter pendiente y nos saltaría la pregunta del nombre. El cin.ignore() limpia ese residuo del buffer garantizando que el getline funcione perfectamente.

**3.Guia de Compilacion y Ejecucion (PowerShell)**

Para compilar y probar el sistema localmente desde Visual Studio Code en Windows, se utiliza el estándar de C++11 con el siguiente comando de compilación limpia:

```
# 1. Compilación
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe

# 2. Ejecución
.\src\piedrapapelotijera.exe
```

Estas líneas de comando son las que se deben colocar en la Terminal de la carpeta raíz, para que el compilador g++ (compilador de lenguaje C#, C+ o C++) pueda leer correctamente el programa y ejecutarlo con la segunda.